

CSCE 669 COMPUTATIONAL OPTIMIZATION · MODULE 05 · LECTURE NOTES

First-Order Methods

Gradient descent, with rates.

MODULE OUTCOMES

- State convergence rates for the standard assumptions.
- Explain the condition number's role.
- Explain acceleration and what it achieves.
- Apply proximal methods to non-smooth objectives.
- Connect the theory to CSCE 636's practice.

1 Convergence rates

Assumptions	Rate	Iterations to reach ϵ
Convex and L-smooth	$O(1/k)$	$O(1/\epsilon)$ — sublinear. Halving the error doubles the work.
Convex, L-smooth, accelerated	$O(1/k^2)$	$O(1/\sqrt{\epsilon})$ — and this is optimal (§2).
Strongly convex ($\mu > 0$) and smooth	$O((1 - 1/\kappa)^k)$	$O(\kappa \log(1/\epsilon))$ — linear convergence . Each additional digit of accuracy costs a constant number of iterations rather than a multiplying factor.
Strongly convex, accelerated	$O((1 - 1/\sqrt{\kappa})^k)$	$O(\sqrt{\kappa} \log(1/\epsilon))$
Convex, non-smooth	$O(1/\sqrt{k})$	$O(1/\epsilon^2)$ — substantially worse, which motivates §3.
Non-convex, smooth	$O(1/\sqrt{k})$ to a stationary point.	No global guarantee of any kind — only that the gradient norm shrinks. CSCE 636's situation.

Strong convexity is what buys linear convergence, and the jump from $O(1/\epsilon)$ to $O(\log(1/\epsilon))$ is by far the largest gain in this table — the difference between a million iterations and twenty for six digits of accuracy. **Which means adding a small L2 term to make a problem strongly convex can be worth it for the optimisation alone**, independently of its regularisation effect (CSCE 633 Module 04).

THE CONDITION NUMBER DECIDES THE SPEED

$\kappa = L / \mu$ — the ratio of the largest to the smallest curvature of the objective. **For a quadratic it is exactly the ratio of the largest to the smallest eigenvalue of the Hessian**, and it is the same condition number as in numerical linear algebra (CSCE 633 Module 02 §1).

A large κ means a long thin valley, and gradient descent zigzags across the narrow direction while crawling along the long one — **which is exactly the picture CSCE 633 Module 02 §2 drew to motivate feature scaling**, now with a quantity attached to it.

The iteration count scales linearly with κ , or with $\sqrt{\kappa}$ when accelerated. So a condition number of 10,000 means ten thousand iterations rather than a hundred — a difference that is visible as 'the model will not train'.

So conditioning is worth attacking directly, and it is actionable: **feature scaling, preconditioning, batch normalisation, and L2 regularisation all reduce κ . Which explains why regularisation speeds up optimisation as well as improving generalisation** — two benefits from one term, for two unrelated reasons, which is a coincidence worth knowing about.

2 Acceleration

```
HEAVY BALL (Polyak)
  v = beta*v + grad f(x)
  x = x - eta*v
  accumulate a velocity; damps oscillation across the valley

NESTEROV
  y = x + beta*(x - x_prev)      # look ahead FIRST
  x = y - eta * grad f(y)       # gradient AT the lookahead
  the gradient is evaluated at the EXTRAPOLATED point. That one
  change gives the  $O(1/k^2)$  rate.

WHY IT MATTERS
   $O(1/k^2)$  is OPTIMAL for first-order methods on smooth convex
  problems -- a LOWER BOUND, proved by Nemirovski and Yudin.
  No method using only gradient information can do better.

  SO ACCELERATION IS NOT A HEURISTIC. It attains a known limit.

WHAT IT IS NOT
  it is not a descent method -- the objective can INCREASE on
  individual steps. Expected, and alarming to watch.
```

That acceleration attains a proved lower bound is a much stronger statement than 'momentum helps', and it is the clearest distinction between this course and CSCE 636 Module 03: there, momentum was a technique that worked; here it is known to be optimal and known to be unimprovable without second-order information (Module 07).

3 Non-smooth objectives

PROXIMAL METHODS HANDLE THE NON-SMOOTH PART EXACTLY

Many objectives split as $f + g$, where f is smooth and g is non-smooth but structurally simple — least squares plus an ℓ_1 penalty being the standard case (CSCE 633 Module 04).

Subgradient descent handles it and is slow — $O(1/\epsilon^2)$ by §1, which is the worst row in the table, and it discards the fact that f was perfectly smooth.

Proximal gradient does much better: take an ordinary gradient step on the smooth part f , then apply the *proximal operator* of g — which solves g 's contribution exactly rather than approximating it by a subgradient. **The rate returns to $O(1/k)$, and to $O(1/k^2)$ when accelerated (FISTA).**

And for the ℓ_1 penalty the proximal operator is soft thresholding: shrink each coordinate toward zero by a fixed amount and clip at zero. **Which is precisely why the lasso produces exact zeros** (CSCE 633 Module 04 §1) — **the geometric corner argument there is now an algebraic fact about an operator**, derived rather than asserted, and the two explanations agree.

4 Back to practice

Practice from CSCE 636	What the theory says	Status
Feature scaling and normalisation	Reduces κ , so fewer iterations (§1).	Fully explained.
Momentum	Attains the optimal first-order rate (§2).	Explained, for convex problems. The deep learning case is analogous rather than covered.
L2 regularisation / weight decay	Adds $\mu > 0$, making the problem strongly convex and the convergence linear.	Explained — and it predicts that weight decay speeds training, which it does, for a reason unrelated to generalisation.
Adam	Approximately diagonal preconditioning, which reduces the effective condition number per coordinate.	Partly. Convergence proofs for Adam are delicate and the original one was wrong — AMSGrad was proposed to fix it.
Why SGD finds minima that generalise	Nothing.	Open (CSCE 636 Module 03 §4). The theory has nothing to say, and the flat-minima explanations remain contested.
Learning rate schedules	Some results for convex problems; the explore-then-exploit framing is not derived.	Mostly empirical.

WHY THE CONVEX THEORY IS STILL WORTH HAVING

It tells you what is achievable. The lower bounds of §2 say that no first-order method does better than $O(1/k^2)$ — so a slow run is a **conditioning problem or a problem-structure problem, not a matter of finding a cleverer optimiser**. That redirects effort correctly.

It identifies the quantity to attack. κ is measurable and actionable: scale the features, precondition, add a regularisation term, reformulate. **Without the theory there is no named quantity to improve**, only a slow run.

It explains why the practical tricks work, which is what lets you predict when they will *not* — a technique justified only by having worked before transfers unpredictably.

And many sub-problems inside non-convex pipelines are convex: the final linear layer given fixed features, the proximal step, the line search, the dual problem (Module 04 §1 — always convex). **So the theory applies locally even where it does not apply globally**, which is more often than the non-convexity of the overall problem suggests.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Nesterov — Introductory Lectures on Convex Optimization	The rates of §1 and the lower bounds of §2 , from the person who proved most of them. Dense; the first two chapters are the relevant ones.
Boyd — EE364B, subgradient and proximal methods (free)	The §3 material , including the proximal operator catalogue and the soft-thresholding derivation.
Beck & Teboulle — FISTA (free)	Accelerated proximal gradient — §2 and §3 combined, and the standard method for the lasso.
Sebastian Bubeck — Convex Optimization: Algorithms and Complexity (free)	A free, modern, and readable treatment of this whole module , with the lower bounds stated clearly.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Verify the $O(1/k)$ rate empirically** on a smooth convex problem by plotting error against iteration on log axes.
- 2 **Add an L2 term and verify the rate becomes linear.**

- 3 **Construct quadratics with condition numbers 10, 100, and 1000** and measure the iteration count for each.
- 4 Confirm the iteration count scales with κ as predicted.
- 5 Implement heavy ball and Nesterov acceleration and compare against plain gradient descent.
- 6 **Verify the $O(1/k^2)$ rate** and show the objective increases on some steps.
- 7 **Derive the proximal operator of the L1 norm** and confirm it is soft thresholding.
- 8 **Implement proximal gradient for the lasso** and compare its convergence against subgradient descent.
- 9 Precondition a badly conditioned problem and measure the improvement.
- 10 **Measure the condition number of a problem from CSCE 633** and predict its iteration count.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give convergence rates under six sets of assumptions.
- 2 What does strong convexity buy, and why is it the largest gain?
- 3 Define the condition number and say what reduces it.
- 4 Describe Nesterov acceleration and what distinguishes it from heavy ball.
- 5 Why is acceleration not a heuristic?
- 6 Why is acceleration not a descent method?
- 7 What is a proximal operator, and what is the L1 one?
- 8 What does the theory explain about CSCE 636's practice, and what does it not?